Ограниченный контекст — не значит ограниченная информация: структурированное состояние покупает компактность промпта ценой решаемости задач у кодирующих агентов с длинным горизонтом

Alex Markov-Buturskiy

Draft 21 сентября 2026 г.

Аннотация

Структурированное состояние — это попытка заменить историю диалога агента компактным блоком, который модель сама ведёт и перезаписывает. На синтетических бенчмарках с длинным горизонтом такая замена держит промпт почти постоянного размера и, судя по опубликованным цифрам, не портит качество решения задач. Работает ли этот эффект на настоящем программировании, а не только на синтетике? Мы проверяем это на Terminal-Bench 2.0, с дизайном, зафиксированным письменно до какого-либо запуска (§4): 87 задач $\times$ 4 условия $\times$ 3 испытания $=$ 1044 запусков, без пропущенных ячеек, и вердиктами, взятыми только из собственного верификатора каждой задачи.

Эффект не воспроизводится. Метод в опубликованном виде решает 0.349 задач против 0.613 у контроля с полным транскриптом; в попарном сравнении по (задаче, испытанию) он проигрывает 82 задач и выигрывает лишь 13 ($p = 2.1 \times 10^{-13}$). Хуже того, потеря *растёт* именно на длинных задачах (54 против 8, $p = 1.7 \times 10^{-9}$) — там, где память как раз должна была помогать больше всего. Если рядом с блоком состояния оставить шесть дословных обменов, разница с контролем исчезает (29 против 23 несовпадающих задач, $p = 0.49$): значит, урон причиняет именно выброшенная история, а не сам блок состояния. Мы формулируем это как неспособность обнаружить разницу, а не как доказательство их равноценности.

Сами трассы запусков показывают механизм напрямую. Ограничить *промпт* — не то же самое, что ограничить *доступ к информации*: агент довыясняет выброшенный контекст через собственные инструменты, повторяя побайтово идентичные команды осмотра в 58 % всех команд у провалившихся run-ов с одним лишь состоянием против 20 % у контроля, и в 12 раз чаще переходя по указателям к файлам. Цикл инструментов доплачивает ходами за то, что промпт якобы сэкономил на контексте. Экономика следует из этого напрямую: при одинаковом результате рука с одним состоянием несёт 0.47$\times$ от пикового промпта контроля, но обходится в 2.57$\times$ дороже — поддержание состояния тарифицируется по цене вывода (4.55$\times$), а блок, переписываемый на каждом ходу, ломает кэш префикса, которым свободно пользуется контроль (352.2M кэшированных токенов против 6.2M).

Проверенный режим — одно семейство моделей на одном бенчмарке, и в этом режиме контроль ни разу не подходит вплотную к пределу окна контекста. Поэтому эти цифры показывают, что даёт механизм, когда окно *не* является узким местом, а не то, что он даёт, когда является.

Вклад этой работы: опровержение переносимости заявленного эффекта на реальных задачах программирования; указание на свойство исходной оценки, которое это скрывало (фиксированное число обращений к модели за эпизод по построению не даёт агенту компенсаторно довыяснять информацию); метод учёта стоимости с поправкой на кэш, показывающий, что сокращение токенов и сокращение затрат — разные вещи; и открытый датасет из 1044 полностью инструментированных траекторий агента.

Как читать пометки источника. Каждое утверждение о нашей системе помечено одним из трёх

значков: [live] значит измерение на реальном запуске, [code] — свойство кода, [orig] — утверждение из исходных статей. Без пометки — это довод, а не свидетельство.

1 Введение

Кодирующий агент, работающий часами, копит у себя за спиной необъятный транскрипт: каждое наблюдение, каждый вывод инструмента, каждая промежуточная мысль при следующем обращении к модели отправляются заново. Промпт растёт вместе с задачей, а стоимость хода — вместе со всем, что ему предшествовало. Структурированное состояние — самый прямой ответ на эту проблему: выбросить транскрипт и заставить модель вместо него вести компактный, структурированный блок состояния, которого должно хватать, чтобы знать, что делать дальше. По данным skill.state [Badhe et al., 2026], такая замена удерживает размер промпта почти постоянным при росте среды от 10 до 200 событий, и при этом качество решения задач не падает[orig].

Этот результат измерен на синтетическом генераторе задач. Мы спрашиваем, переживает ли он столкновение с настоящим программированием, где сам агент решает, что смотреть и когда работа закончена.

Что именно мы проверяем. Не абстрактную «полезность» структурированного состояния, а конкретное утверждение о переносимости, вытекающее из опубликованной оценки: замена транскрипта поддерживаемым блоком состояния не роняет способность решать задачи при том же ограничении на контекст. Проверяем на Terminal-Bench 2.0, с дизайном, зафиксированным письменно до первого запуска (§4), и вердиктами, которые берём только из собственного верификатора каждой задачи.

Результат. Утверждение не подтвердилось. Из 1044 запусков опубликованная конфигурация решает 0.349 задач против 0.613 у контроля с полным транскриптом, проигрывая 82 парных задач и выигрывая лишь 13 ($p = 2.1 \times 10^{-13}$). Это не следствие трудного набора задач или слабой модели: та же модель на тех же задачах с нетронутой историей справляется без проблем.

Два дополнительных наблюдения превращают отрицательный результат в объяснение. Во-первых, потеря *растёт* вместе с длиной задачи (§7) — ровно наоборот тому, что обещает метод: его выгода должна была накапливаться именно на длинных задачах. Во-вторых, если вернуть блоку состояния всего шесть дословных реплик из истории, разница с контролем перестаёт обнаруживаться ($p = 0.49$; это отсутствие доказательства, а не доказательство равенства). Значит, урон нанесла именно выброшенная история, а не сама идея состояния.

Почему исходная оценка не могла это заметить. В синтетическом дизайне число обращений к модели за эпизод фиксировано по построению: среда сама порождает T событий, и рантайм отвечает на каждое ровно одним обращением[orig]. Забывчивому агенту там попросту *не на что* тратить лишние ходы — в конструкции нет канала, через который повторное выяснение информации могло бы проявиться как затрата. У реального агента число шагов — результат, а не заданный параметр, и именно тут появляется цена (§6).

Механизм, сформулированный как тезис статьи.

> *Ограничить контекст беседы — не значит ограничить доступную агенту информацию: кодирующие агенты компенсируют потерю, повторно осматривая среду.*

Блок состояния ограничивает то, что агенту *сообщают*; он не ограничивает то, что агенту *нужно знать*. Разница оплачивается через цикл вызовов инструментов, за которым не следит ни одна метрика размера промпта. Мы измеряем это напрямую: в провалившихся запусках с одним лишь состоянием 58 % всех команд — это повтор уже выполненного осмотра, против 20 % у контроля[live] (§8).

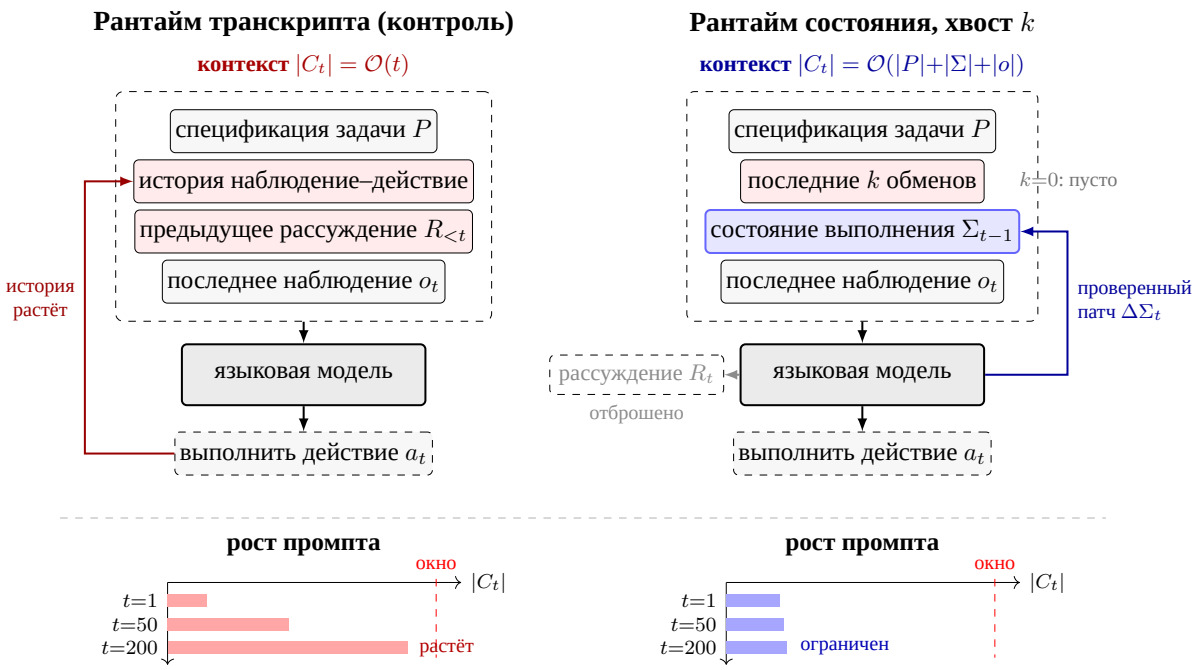


Рис. 1: Две политики контекста из Алгоритмов 1 и 2. **Слева:** рантайм транскрипта дополняет каждый обмен, так что промпт растёт вместе с эпизодом (Ур. 1) и в итоге упирается в окно контекста. **Справа:** рантайм состояния несёт проверенный блок состояния и, при k=0, вообще никакой истории (Ур. 3); его промпт ограничен независимо от t. Оба отбрасывают промежуточное рассуждение. Ограниченный промпт — это опубликованная выгода. Чего диаграмма не может показать — это величина, которую измеряет данная статья: ограничение $|C_t|$ не ограничивает информацию, требуемую задачей, так что агент заново её получает через вызовы инструментов, раздувая реализованный горизонт (Ур. 6) без затрат, видимых какой-либо метрике размера промпта.

Экономика следует из того же механизма. Раз состояние выдаётся моделью как сгенерированный текст, оно тарифицируется по цене вывода; раз блок переписывается на каждом ходу, он разрушает кэш префикса, которым почти целиком обслуживается растущий только по дополнению транскрипт. При одинаковом результате рука с одним состоянием несёт 0.47× от пикового промпта контроля и стоит 2.57× дороже[live]. Меньше токенов — не значит дешевле, а на этом корпусе даже не значит дешевле в ту сторону, куда указывает подсчёт токенов.

Вклад работы.

1. Опровержение, на закрытом предрегистрированном дизайне из 1044 запусков, утверждения о том, что структурированное состояние сохраняет способность решать реальные программные задачи (§7).
2. Свойство исходной оценки, которое это скрывало: фиксация числа обращений к модели за эпизод по построению не даёт компенсаторному довыяснению информации проявиться в измерениях (§6).
3. Механизм, измеренный на уровне отдельных вызовов инструментов, напрямую связывающий выброшенную историю с избыточным осмотром среды (§8).
4. Методика учёта стоимости с поправкой на кэш, которая показывает, что сокращение токенов и сокращение денежных затрат — разные величины, и что ни одна конфигурация структурированного состояния не выходит дешевле транскрипта при равном результате (§9).
5. Открытый датасет из 1044 полностью инструментированных траекторий с происхождением каждого запуска и детерминированным анализом, который воспроизводит каждое число этой статьи (§A).

Чего мы не утверждаем. Мы не утверждаем, что структурированное состояние — плохая идея. Мы утверждаем, что его опубликованная оценка измеряет режим, в котором его главный отказ попросту не может произойти, и что на реальных программных задачах этот механизм покупает компактность промпта ценой решаемости — по курсу, который исходная метрика не способна выразить. Раздел 11 прямо называет границы: одно семейство моделей, один корпус задач, и исследование композиции, которое всё ещё продолжается.

2 Предпосылки

2.1 Память как структурированное состояние

skill.state [Badhe et al., 2026] ведёт постоянное структурированное состояние Σ_t и собирает каждый промпт как

$$\text{промпт}_t \;=\; [\,\text{спецификация задачи}\,] \;+\; [\,\Sigma_t\,] \;+\; [\,o_t\,],$$

где o_t — последнее наблюдение, а история взаимодействия выбрасывается. Модель за одну генерацию выдаёт рассуждение, патч состояния и действие — $(R_t, \Delta\Sigma_t, a_t)$ — единым JSON-объектом с ровно двумя ключами, `state_patch` и `action`[orig]. Рантайм проверяет каждый патч по схеме и откатывает-повторяет невалидные. Схема пишется один раз на предметную область; на 100 задачах InterCode CTF используется одна и та же схема с пятью полями[orig].

Опубликованное свидетельство — это горизонтная развёртка на процедурном генераторе: размер промпта держится около 1800 символов от $T{=}10$ до $T{=}200$, тогда как у базовых линий он растёт до 48,000, а качество сохраняется или улучшается[orig]. Собственная таксономия ошибок авторов для моделей с открытыми весами относит 68 % отказов на счёт преждевременной перезаписи или удаления состояния[orig]— к этой цифре мы вернёмся в §10, потому что она сама по себе довод в пользу того, чтобы забрать часть состояния из рук модели.

2.2 Детерминированное состояние исполнения

ledger [Wang et al., 2026] занимает противоположный угол конструкторского пространства: запись, которую рантайм ведёт механически, из уже завершённых взаимодействий — журналы наблюдений, счётчики изменений по файлам и глобально, нормализованные записи команд вместе с их исходами. Метод не добавляет ни одного обращения к модели и не меняет поведение агента. Два его механизма аккуратно разделяются в собственной абляции авторов: *inform* (вывод записи в конец входа, там, где она не портит append-only префикс) даёт основную часть выигрыша в эффективности, а *govern* (перехват предложенной команды и повторное использование более раннего результата) — основную часть выигрыша в решаемости[orig].

Эти две системы не конкурируют, а дополняют друг друга: skill.state держит смысловое состояние в модели, ledger — объективное состояние в рантайме. Наше исследование композиции (§10) варьирует именно эту границу, и мы сознательно исключаем из него *govern*: govern меняет то, что агент *делает*, и его включение подмешало бы в измерение меньшего эффекта эффект гораздо большего масштаба.

2.3 Проверяемые допущения

Сформулируем их как проверяемые утверждения, а не как критику:

A1 (доступность схемы)

Адекватная предметная схема существует ещё до запуска. Оригинал не даёт метода проектирования схемы и приводит дословный список полей лишь для одной из четырёх предметных областей[orig].

A2 (польза с первого хода)
Ведение состояния в худшем случае нейтрально уже с первого хода. Самый короткий заявленный горизонт — $T{=}10$, что уже длиннее заметной доли настоящих программных задач.

A3 (достаточная статистика)
Ничего из выброшенного больше никогда не понадобится. Оригинал сам называет это ограничением[orig], но не измеряет его. Именно это допущение опровергает данная статья.

M (валюта измерения)
Суммарное число токенов — валидный показатель стоимости. Полнотекстовый поиск по оригиналу не находит ни одного упоминания кэша, биллинга или денежной терминологии[orig].

A1 и A3 — свойства самого метода. M — свойство того, *как* метод оценивали, и именно оно мешает увидеть первые два: метрика, считающая логические токены, не способна отразить ни скидку за кэш, которую вознаграждает append-only префикс, ни вызовы инструментов, потраченные на повторное выяснение того, чего промпт больше не несёт.

2.4 Ось конструирования, использованная в этой статье

Вместо сравнения именованных архитектур мы фиксируем сам механизм и варьируем одно целое число: сколько k последних дословных обменов сохраняется рядом с блоком состояния.

$$\text{промпт}_t \;=\; [\,\text{спецификация задачи}\,] \;+\; [\,\text{последние } k \text{ обменов}\,] \;+\; [\,\Sigma_t\,]$$

При $k{=}0$ это в точности skill.state в опубликованном виде. С ростом k конфигурация приближается к транскрипт-агенту. Контроль — не точка на этой оси (у него вообще нет блока состояния) и всегда приводится отдельно.

Такая ось даёт три вещи, которых не даёт сравнение трёх именованных рукавов. Она убирает произвольную константу, которую незаметно протащил бы промежуточный «гибридный» рукав; превращает вопрос из *какая архитектура* в *где на оси лежит оптимум*; и делает форму зависимости наблюдаемой — а именно форма в конечном счёте и локализует ущерб (§7).

3 Формальная модель

Этот раздел вводит обозначения, записывает оба рантайма в виде алгоритмов и выводит тождество учёта, на которое опирается остальная часть статьи. Асимптотика принадлежит оригиналу[orig]; наш вклад — показать, какой из её членов стандартная оценка держит постоянным, и во что превращается тождество, когда этот член отпускают на свободу.

3.1 Обозначения

Агент проходит задачу за дискретные ходы $t = 1, 2, \ldots$ Пусть P — спецификация задачи, o_t — наблюдение, которое среда возвращает на ходу t, a_t — действие агента, а R_t — его промежуточное рассуждение. Через Σ_t обозначим структурированное состояние выполнения, через C_t — контекст, собранный для обращения к модели на ходу t. Размер в токенах записывается как $|\cdot|$.

Горизонт T — фактическое число ходов эпизода. На различии между входом и выходом для T держится весь §6, поэтому мы помечаем его явно: T^{env}, если его фиксирует среда, T^{ag}, если агент останавливается сам.

3.2 Две политики контекста

Рантайм с транскриптом дополняет каждый обмен, так что его контекст на ходу t равен

$$C_t^{\text{tr}} \;=\; \big[\,P,\; (o_1, R_1, a_1), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\; o_t\,\big], \tag{1}$$

он растёт линейно по t: $|C_t^{\mathrm{tr}}| = \mathcal{O}(t)$, а совокупный объём промпта равен

$$\sum_{t=1}^{T} \bigl|C_t^{\mathrm{tr}}\bigr| \;=\; \mathcal{O}(T^2). \tag{2}$$

Рантайм со структурированным состоянием историю отбрасывает и несёт только состояние:

$$C_t^{\mathrm{st}} \;=\; \bigl[\,P,\ \Sigma_{t-1},\ o_t\,\bigr], \qquad \bigl|C_t^{\mathrm{st}}\bigr| \;=\; \mathcal{O}\bigl(|P| + |\Sigma| + |o|\bigr), \tag{3}$$

если состояние ограничено, эта величина от t не зависит, откуда

$$\sum_{t=1}^{T} \bigl|C_t^{\mathrm{st}}\bigr| \;=\; \mathcal{O}(T). \tag{4}$$

Уравнения (2) и (4) — это опубликованный аргумент; мы воспроизводим его без возражений[orig].

3.3 Семейство хвостов, интерполирующее между ними

Оба случая обобщаются в одно семейство, индексированное целым $k \geq 0$ — числом самых недавних обменов, которые сохраняются дословно рядом с состоянием:

$$C_t^{(k)} \;=\; \bigl[\,P,\ (o_{t-k}, R_{t-k}, a_{t-k}), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\ \Sigma_{t-1},\ o_t\,\bigr]. \tag{5}$$

При $k = 0$ это в точности (3); при $k \to t$ получается (1) с лишним блоком состояния. Контрольная группа блока состояния не несёт вовсе, поэтому в семейство не входит и приводится отдельно на протяжении всей статьи.

Каждое условие статьи — это значение k в (5): сравнение тем самым становится положением на кривой, а не состязанием именованных систем.

3.4 Рантаймы как алгоритмы

Algorithm 1 Рантайм с транскриптом (контрольная группа)

Require: спецификация задачи P

$H \leftarrow \varnothing$ ▷ история взаимодействия
for $t = 1, 2, \ldots$ **do**
  получить наблюдение o_t
  $C_t \leftarrow [\,P,\ H,\ o_t\,]$
  сгенерировать $(R_t, a_t) \sim \pi(C_t)$
  выполнить a_t
  $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
  if агент объявляет завершение **then**
    break
  end if
end for

Algorithm 2 Рантайм состояния с хвостом k

Require: спецификация P; схема S; хвост k

1: $\Sigma_0 \leftarrow \varnothing$; $H \leftarrow \varnothing$
2: **for** $t = 1, 2, \ldots$ **do**
3: получить наблюдение o_t
4: $C_t \leftarrow [\,P,\ \mathrm{tail}_k(H),\ \Sigma_{t-1},\ o_t\,]$
5: сгенерировать $(R_t, \Delta\Sigma_t, a_t) \sim \pi(C_t)$
6: **if** $\mathrm{valid}_S(\Delta\Sigma_t)$ **then**
7: $\Sigma_t \leftarrow \Sigma_{t-1} \oplus \Delta\Sigma_t$
8: **else**
9: $\Sigma_t \leftarrow \Sigma_{t-1}$; вернуть ошибку модели
10: **end if**
11: выполнить a_t; отбросить R_t
12: $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
13: **if** агент объявляет завершение **then**
14: **break**
15: **end if**
16: **end for**

Два рантайма расходятся ровно в одном месте — в том, что собирает строка 4. Слияние $\oplus$ накладывает патч, прошедший проверку по схеме; недействительный патч состояние не трогает. При $k{=}0$

хвост пуст, и Алгоритм 2 — это skill.state в опубликованном виде. Рассуждение R_t отбрасывают оба алгоритма; историю из контекста убирает только Алгоритм 2.

Проверка завершения в обоих случаях (строка 8 Алгоритма 1 и строка 13 Алгоритма 2) — это собственное объявление агента о завершении, а не истечение внешнего счётчика. Это единственное структурное отличие от формулировки оригинала, где цикл выполняется ровно T^{env} раз, и именно это отличие оказывается значимым, как показывает §6.

3.5 Что упускает стандартный учёт

Пусть $\mathcal{I}_t$ — информация, нужная агенту на ходу t, чтобы выбрать правильное действие. Она приходит по одному из двух путей: либо она уже несётся в контексте, либо агент тратит ход на действие осмотра и добывает её заново из среды. Через ρ обозначим долю ходов, потраченных на такое повторное приобретение.

Агент с транскриптом уже заплатил за свою историю — она в промпте, — поэтому ρ^{tr} мало. Агент состояния отбросил всё, чего Σ не кодирует, так что любой нужный факт вне Σ приходится добывать заново, а $\rho^{(k)}$ растёт по мере того, как k уменьшается. Каждое такое приобретение стоит хода, и реализованный горизонт от этого раздувается:

$$T^{\text{ag}}(k) \;\approx\; \frac{T_0}{1-\rho(k)}, \tag{6}$$

где T_0 — число ходов, которого хватило бы агенту с идеальной памятью.

Подставив (6) в (4), получаем величину, которой в асимптотике оригинала попросту нет:

$$\sum_{t=1}^{T^{\text{ag}}} \bigl|C_t^{\text{st}}\bigr| \;=\; \mathcal{O}\!\left(\frac{T_0}{1-\rho}\right), \qquad \rho=\rho(k). \tag{7}$$

Опубликованное сравнение оценивает (4) против (2) при $T = T^{\text{env}}$, одинаковом для всех рук. Этим выбором ρ выносится за пределы модели: раз число ходов фиксирует среда, повторное приобретение не способно удлинить эпизод, и (6) вырождается в $T^{\text{ag}} = T^{\text{env}}$ для любого рантайма — независимо от того, сколько было забыто.

В этом и состоит центральный формальный тезис статьи: асимптотическое преимущество (4) держится на переменной (T), которую оценка удерживает постоянной, тогда как проверяемый механизм (ρ) действует именно на эту переменную. Мы измеряем ρ напрямую в §8, а T^{ag} — в §7.

3.6 Стоимость — это не число токенов

Пусть α, γ, β — цены за токен для некэшированного входа, кэшированного входа и выхода соответственно, а h_t — кэшированная доля промпта на ходу t. Тогда тарифицируемая стоимость эпизода равна

$$\text{Стоимость} \;=\; \sum_{t=1}^{T}\Bigl[\,\alpha\,(1-h_t)\,|C_t| \;+\; \gamma\,h_t\,|C_t| \;+\; \beta\,|y_t|\,\Bigr], \tag{8}$$

где y_t — сгенерированный вывод, который у рантайма состояния включает патч $\Delta\Sigma_t$. У нынешних провайдеров $\gamma \ll \alpha < \beta$.

Отсюда следуют два вывода, и оба измеряются в §9. Контекст транскрипта только дополняется, так что C_t^{tr} продолжает C_{t-1}^{tr}, а $h_t \to 1$: растущий промпт из (2) тарифицируется в основном по дешёвой ставке γ. Блок состояния переписывается на каждом ходу, поэтому префикс расходится начиная с Σ, и h_t остаётся низким даже при маленьком $|C_t|$. Заодно поддержание состояния перекладывает токены из членов $|C_t|$ в $|y_t|$ — а это самая дорогая цена β.

На этом и держится тождество, на котором построена вся работа: минимизировать $\sum_t |C_t|$ и минимизировать (8) — разные задачи, и на измеряемом здесь корпусе у них противоположные решения.

4 Дизайн исследования и заморозка протокола

Этот раздел намеренно стоит перед результатами. Дизайн был зафиксирован письменно до начала серии, и именно последовательность *гипотеза → замороженный метод → данные → интерпретация* даёт читателю возможность отличить находку от истории, собранной задним числом.

4.1 Что было заморожено и когда

Предрегистрация — артефакт репозитория, датированный раньше первого запуска серии[code]. В ней зафиксированы вопрос, гипотезы с условиями их фальсификации, вмешательство, зависимые переменные, вмешивающиеся факторы и контроль над ними, объём выборки с обоснованием мощности, правило остановки, статистические тесты и политика исключений. Раздел поправок в ней пуст: после первого запуска ничего не менялось.

Реальную работу делают три обязательства:

1. **Вердикт внешний.** Исход запуска определяет код завершения собственного верификатора задачи, а не отчёт агента о своём успехе. Запуск без вердикта верификатора в набор данных вообще не попадает.
2. **Тесты выбраны раньше данных.** Точный парный тест Макнемара — для способности решать задачи, точный знаковый тест на попарных отношениях — для стоимости, ранговая корреляция Спирмена — для тренда по горизонту. Ни один тест не подбирался задним числом, после того как эффект уже был замечен.
3. **Исключения определены до того, как увидели подсчёты.** Правило контаминации (§12) и правило схлопывания дубликатов были написаны заранее, до применения. Второе оставляет в выборке *самый ранний* запуск повторно запущенной ячейки, так что повтор не может протолкнуть себя в выборку победой.

4.2 Гипотезы и условия их опровержения

H1 (стоимость).
Медианное отношение стоимости только-состояния к контрольной группе превышает 1.0. *Опровергается*, если отношение ≤ 1.0, знаковый тест не достигает значимости, или направление расходится между тремя испытаниями.

H2 (горизонт).
С ростом горизонта отношение стоимости падает и пересекает 1.0: накладные расходы амортизируются на длинных задачах. *Опровергается*, если корреляции нет, она положительна, или ни разу не приближается к 1.0 в наблюдаемом диапазоне.

H3 (способность).
Приводится с доверительным интервалом; считается недостаточно мощной, если наблюдаемое несовпадение не превышает предвычисленное требование в шесть односторонних несовпадающих пар.

H2 — гипотеза, которую это исследование опровергает предрегистрированным образом; мы сообщаем об этом прямо в §9, а не отодвигаем в сторону.

4.3 Множественные сравнения

Предрегистрированы были три гипотезы, и именно они несут подтверждающий вес исследования. Всё остальное, для чего статья приводит p-значение — разбивки по стратам, ступени лестницы §10, сравнения стоимости по условиям, — вторично и разведочно, и поправка на семейственность сравнений к этому не применяется.

Мы проговариваем это явно, потому что §B критикует оригинал именно за отсутствие такой оговорки, а стандарт должен быть один для обеих статей. Следствие конкретно: ключевой результат переживает любую поправку, которую к нему можно применить ($p = 2.1 \times 10^{-13}$ по основному сравнению против порога Бонферрони $0.05/3$ для подтверждающего семейства), а вот несколько вторичных результатов лежат достаточно близко к 0.05, чтобы её не пережить. Там, где вторичный результат несёт вес для утверждения статьи, мы говорим об этом прямо и трактуем его как предварительный, а не установленный.

4.4 Заранее объявленная ловушка артефакта

Одна ловушка была названа заранее именно потому, что она грозила приукрасить результат: *более низкая* стоимость на длинных задачах в руке только-состояния могла бы означать не амортизацию, а всего лишь более частое исчерпание бюджета ходов — то есть цензурирование. Поэтому предрегистрация исключает запуски, остановленные по бюджету, из агрегации стоимости и приводит их число по каждой страте отдельно. Ловушка оказалась реальной: распределение причин остановки в Таблице 4 показывает остановки по бюджету, сконцентрированные именно в этой руке.

4.5 Охват

Эта серия отвечает на вопросы о стоимости и горизонте и приводит способность решать задачи. Чувствительность к схеме и композиция состояния — отдельные дизайны с собственными предрегистрациями; их статус дан в §10. [**PENDING:** если владелец решит опубликовать только E1, этот абзац станет опережающей ссылкой на вторую статью]

5 Экспериментальная установка

5.1 Агент, корпус и условия

В качестве агента используется немодифицированный кодирующий агент с открытым исходным кодом; механизм состояния реализован плагином, который подменяет собираемый для каждого обращения к модели контекст и добавляет проверенный по схеме инструмент патчей[code]. Версии, хэш содержимого плагина и хэш схемы зафиксированы на всю серию и записаны в каждой строке запуска (плагин `be0f5b73e526ba4a`, схема `7d8705aae4e124f1`)[live].

Корпус — Terminal-Bench 2.0. Из 89 задач, в принципе выполнимых, стенд реально достигает 87: две отказывают при настройке контейнера ещё до того, как агент приступает к работе, причём при каждой попытке и в каждом условии[live]. Модель на протяжении всего исследования одна — `zai/glm-5.3-flash`.

Четыре условия различаются ровно одним свойством:

Условие	Контекст, собираемый для каждого обращения к модели
control	полный транскрипт, без блока состояния (немодифицированный агент)
state-only	блок состояния заменяет транскрипт ($k{=}0$)
state+2	блок состояния + 2 самых последних обмена
state+6	блок состояния + 6 самых последних обменов

Всё остальное заморожено — модель, схема, инструменты, бюджет ходов, задача и верификатор одни и те же. Реализованное значение k считывается обратно из трассы каждого запуска; если оно расходится с запрошенным, запуск отклоняется, а не анализируется[code].

5.2 Единица, парность и повторение

Экспериментальная единица — один запуск одной задачи при одном условии в одном испытании. Каждое сравнение парное внутри *(задачи, испытания)* против контрольной группы того же испытания и никогда — против объединённой контрольной группы: у единичных запусков идентичной ячейки число обращений к модели измерялось равным то 15, то 51, то 65[live], так что объединение между испытаниями означало бы сравнивать условие с другой выборкой того же шума. Три независимых испытания дают в сумме 1044 запусков.

5.3 Вердикты

Вердикт — это код завершения собственного верификатора задачи, и это не формальность: на более ранней фазе проекта самоотчёт модели вместе со сломанным контейнером верификатора произвели целую серию строк, похожих на данные, но данными не бывших (§12).

Запуски дополнительно классифицируются по способу завершения (output_ceiling, solved, turn_budget, unsolved), и это распределение приведено в Таблице 4: засчитать остановку по бюджету или по потолку вывода как обычный провал значило бы приписать свойство харнеса самому методу.

5.4 Учёт стоимости

Использование по вызовам — входные, кэшированные для чтения и выходные токены — берётся из журнала сессии. Стоимость выводится из этих измеренных токенов по одной фиксированной опубликованной таблице цен, а не читается у провайдера, и на то есть две измеренные причины: два кошелька одного аккаунта тарифицируют идентичное число токенов ровно вдвое поразному[live], а путь через кастомный эндпоинт сообщает нулевую стоимость, хотя возвращает реальные токены[live]. Собственная цифра провайдера при этом сохраняется в сырых записях для аудита.

Кэшированные для чтения токены отслеживаются отдельно от входных на протяжении всего исследования: центральное методологическое утверждение состоит именно в том, что это разные валюты. Перед серией мы проверили, что кэш префикса провайдера действительно виден в полях использования: повторный вызов сдвинул счётчик с 0 до 3008 кэшированных токенов из 3019[live].

5.5 Проверки перед любым измерением

Прежде чем довериться изменённому стеку, его прогоняют через дымовое тестирование[code]:

1. эталонное решение проходит верификатор — это устанавливает, что верификатор вообще способен распознать правильный ответ;
2. каждая рука демонстративно загружает плагин и реализует заявленное значение k, что проверяется по собранному промпту, а не по собственному логу плагина: молчаливый сбой импорта иначе незаметно превращает экспериментальную руку в контрольную группу под другим именем, а числа при этом продолжают выглядеть правдоподобно;
3. версии и хэши содержимого зафиксированы и записаны для каждого запуска.

Проверка 2 поймала в этом проекте реальный дефект стейджинга[live]— именно поэтому она смотрит на промпт, обращённый к провайдеру, а не на внутреннюю трассу.

6 Воспроизведение: что способен и не способен выразить исходный дизайн

Прежде чем измерять расхождение, стоит убедиться, что с собственными числами оригинала расхождения нет.

6.1 Опубликованное преимущество переживает учёт с поправкой на кэш

Мы пересчитываем итоговую таблицу оригинала по нашей модели стоимости: их приведённые числа токенов, наши измеренные тарифы за токен и наши измеренные показатели попадания в кэш для двух архетипичных рантаймов. Результат сужает их преимущество, но не переворачивает его^{live}:

Горизонт T	отношение токенов	отношение стоимости при учёте кэша
10	1.6×	**0.97**×
25	2.9×	1.75×
50	5.7×	3.42×
100	19.0×	11.45×
200	21.3×	12.82×

Отсюда два вывода. Кэширование сокращает преимущество примерно вдвое, но не инвертирует его: внутри исходного дизайна метод остаётся выигрышным по стоимости, и мы это признаём. А вот при $T{=}10$ отношение уже пересекло 1.0: паритет по стоимости исчезает уже на самом коротком горизонте их собственной таблицы, и остаётся незамеченным лишь потому, что их метрика этого попросту не видит. Это A2, фальсифицированное на опубликованных данных, без единого нового запуска.

Эта оценка переносит наши тарифы на их токены, и мы это оговариваем: порядок величины держится и на их приведённых числах, но точные множители — не измерение их реальных расходов.

6.2 Свойство, скрывающее повторное приобретение

В исходном генераторе горизонт T — это подсчёт событий среды, и каждый рантайм отвечает на каждое событие ровно одним обращением к модели^{orig}. Число обращений к модели тем самым зафиксировано по построению и одинаково во всех руках.

Именно это единственное свойство и определяет, что вообще способна увидеть оценка. Доминирующая стоимость отбрасывания истории — это повторное приобретение отброшенного, а повторное приобретение тратится на *дополнительные шаги, которые агент инициирует сам*. В дизайне, где число шагов — вход, а не выход, этой стоимости просто негде проявиться. Агент, забывший факт и вынужденный заново его вывести, всё равно получает ровно T обращений; забывание в журнале, который ведёт статья, невидимо.

В агентной постановке число шагов — уже результат. На нашем корпусе рука только-состояния тратит 2.22× от числа ходов контрольной группы на задачах, которые решили *обе* руки^{live}, и это отношение измерено только там, где метод преуспел — то есть это нижняя граница накладных расходов, а не среднее по его провалам.

6.3 Что это подтверждает, а что нет

Механика оригинала воспроизводится, его утверждение о токенах воспроизводится, а знак преимущества переживает учёт кэша *внутри его собственного дизайна*. Не переносится вывод из этого дизайна на агентное выполнение. Обе оценки легитимны и отвечают на разные вопросы — просто только один из них тот, что практик задаёт себе перед тем, как встраивать архитектуру памяти в кодирующего агента.

Есть одна оговорка о точности воспроизведения, которую лучше назвать здесь, чем спрятать: наш блок состояния на шаге толще, чем цифра, приведённая в оригинале^{live}, так что наши абсолютные величины стоимости — верхние границы. Более тонкий блок усилил бы сокращение размера промпта и ослабил инверсию стоимости; оба эффекта работают против нашего ключевого результата, и поэтому мы приводим оба. [**NOTE:** рука верности исходной статье, которая это

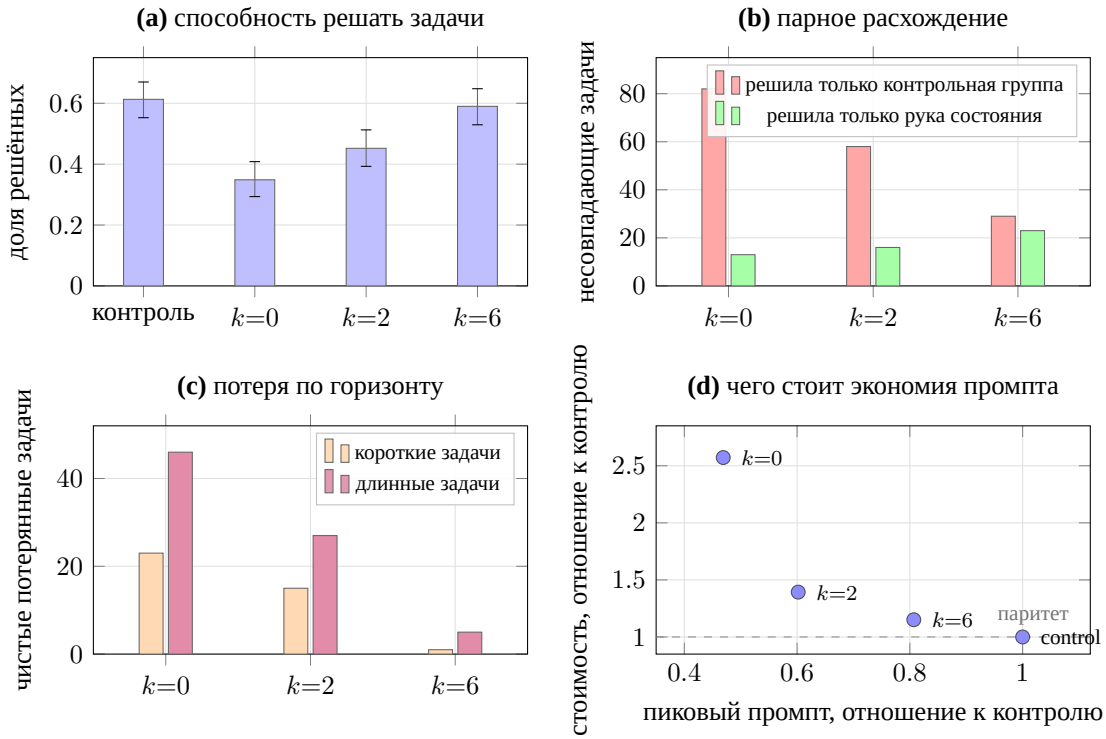


Рис. 2: Характеристика памяти со структурированным состоянием на 87 программных задачах, 1044 запусков. **(a)** Доля решённых задач с интервалами Уилсона; контрольная группа решает 0.613 против 0.349 для опубликованной конфигурации. **(b)** Парное расхождение против контрольной группы того же испытания — только несовпадающие задачи несут информацию, и они подавляюще односторонни при $k{=}0$. **(c)** Чистые потерянные задачи, разбитые по медианному горизонту контрольной группы: дефицит *расширяется* на длинных задачах, инвертируя аргумент в пользу метода. **(d)** При равном исходе опубликованная конфигурация вдвое сокращает пиковый промпт и умножает стоимость на 2.57. Восстановление шести обменов ($k{=}6$) устраняет дефицит способности решать задачи и сокращает премию по стоимости с 2.57$\times$ до 1.15$\times$, но не опускается ниже контрольной группы: при равном исходе она остаётся более дорогой рукой ($p = 0.04$).

изолирует, построена и предрегистрирована; если она запустится до подачи статьи, её числа заменят этот абзац]

7 Результаты I: способность решать задачи

Рисунок 2 обобщает этот раздел и два следующих; таблицы ниже приводят те же величины в точности.

7.1 Доля решённых задач

Таблица 1 даёт непарную картину. Опубликованная конфигурация решает 0.349 запусков против 0.613 у контрольной группы, и интервалы не пересекаются. По мере восстановления истории доля растёт монотонно: 0.452 при двух обменах, 0.590 при шести.

7.2 Парное сравнение

Непарные доли недооценивают дизайн: каждое условие прошло каждую задачу, а значит информативна не сама доля, а множество задач, на которых две руки расходятся. Таблица 2 сравнивает каждую руку с контрольной группой того же испытания.

Таблица 1: Доля решённых задач по условиям. 1044 запусков, 87 задач $\times$ 3 испытаний. Интервалы — по Уилсону.

Condition	Solved / runs	Rate	95% CI
Control (full transcript)	160 / 261	0.613	[0.553, 0.670]
State-only (k=0)	91 / 261	0.349	[0.293, 0.408]
State +2	118 / 261	0.452	[0.393, 0.513]
State +6	154 / 261	0.590	[0.529, 0.648]

Таблица 2: Парная способность решать задачи против контрольной группы того же испытания, точный тест Макнемара. Информацию несут только несовпадающие ячейки.

Condition	Pairs	Control only	Arm only	Both	p
State-only (k=0)	261	82	13	78	2.1×10^{-13}
State +2	261	58	16	102	9.7×10^{-7}
State +6	261	29	23	131	0.49

Опубликованная конфигурация проигрывает 82 задач и выигрывает 13 ($p = 2.1 \times 10^{-13}$). Эффект велик, односторонен и далеко за порогом значимости. При двух сохранённых обменах дефицит всё ещё виден (58 против 16, $p = 9.7 \times 10^{-7}$), а при шести пропадает вовсе: 29 против 23 неотличимы от шума ($p = 0.49$).

Эта последняя строка — самая информативная во всей статье. Шесть дословных обменов — небольшая добавка к промпту, но её достаточно, чтобы вернуть способность решать задачи к паритету. Значит, ущерб наносят не суммирование состояния и не его структурирование, а именно *отбрасывание недавнего транскрипта*: то, что блок состояния отнимает у способности решать задачи, компенсирует история, которую он заменяет, а не сам блок состояния.

7.3 Горизонт: потеря растёт там, где обещали выгоду

Задачи разбиты по медианному числу ходов контрольной группы (11 ходов), что даёт 45 коротких и 42 длинных задач. Горизонт берётся из *контрольной* руки: собственное число ходов экспериментальной руки использовать было бы циклично, поскольку рука, заново добывающая данные, по построению делает больше ходов.

У опубликованной конфигурации дефицит на коротких задачах составляет 28 против 5 ($p = 6.6 \times 10^{-5}$) и расширяется до 54 против 8 на длинных ($p = 1.7 \times 10^{-9}$).

Это переворачивает аргумент в пользу метода. Структурированное состояние мотивируют именно длинными горизонтами, на которых неограниченный транскрипт якобы становится неуправляемым. На этом корпусе всё наоборот: длинный горизонт — там, где отбрасывание транскрипта обходится дороже всего, так что дефицит нарастает по ходу задачи в том же направлении, в котором ожидалась выгода.

При k=6 различий не показывает ни одна страта, а значит эффект горизонта — свойство того, сколько истории было удалено, а не свойство самого блока состояния.

7.4 Как завершались запуски

Сравнение способности решать задачи честно лишь тогда, когда ограничение харнеса не засчитывают как неспособность агента. Таблица 4 разделяет четыре терминальных состояния.

Исчерпание бюджета ходов встречается только в руках, несущих состояние, и никогда — в контрольной группе или при k=6^{live}. Это само по себе свидетельство в пользу механизма из §8: рука, вынужденная заново получать свой контекст, — это та же рука, у которой заканчиваются ходы.

Таблица 3: Парная способность решать задачи по страте горизонта, разбитой по медиане контрольной группы в 11 ходов.

Condition	Horizon	Control only	Arm only	p
State-only ($k{=}0$)	short	28	5	6.6×10^{-5}
State-only ($k{=}0$)	long	54	8	1.7×10^{-9}
State $+2$	short	23	8	0.01
State $+2$	long	35	8	4.2×10^{-5}
State $+6$	short	11	10	1.00
State $+6$	long	18	13	0.47

Таблица 4: Причины остановки по условиям. Остановки по бюджету сконцентрированы в руке, занимающей больше всего ходов, поэтому они исключены из агрегации стоимости.

Condition	output_ceiling	solved	turn_budget	unsolved
Control (full transcript)	62	160	0	39
State-only ($k{=}0$)	117	91	34	19
State $+2$	95	118	18	30
State $+6$	67	154	0	40

Это же и заранее объявленная ловушка артефакта из §4, и именно поэтому каждое утверждение о стоимости в §9 вычисляется только по парам, где задачу действительно решили обе руки.

8 Результаты II: механизм

Раздел 7 показывает, что отброшенная недавняя история обходится в решаемость задач. Этот раздел показывает *как* именно платится эта цена — по вызовам инструментов тех же самых запусков.

8.1 Агент сам выкупает то, что вычеркнул промпт

Запуск с одним лишь состоянием проваливается не потому, что забывает план. Он проваливается потому, что заново выводит то, что уже успел пронаблюдать. Разбираем по назначению каждую команду оболочки из 472 полностью трассированных запусков и считаем команды, повторяющие более ранний идентичный вызов[live]:

Условие	Команд на запуск	Доля осмотра	**Повторный осмотр**
control (полная история)	95	0.23	**0.20**
state-only, решено	163	0.52	0.45
state-only, провалено	278	0.67	**0.58**

У провалившегося запуска с одним состоянием 58 % всех команд уходит на повтор уже выполненного осмотра — против 20 % у контроля. На отдельной матрице тот же эффект виден и по другому показателю: переходы по указателям к файлам случаются 5.19 раза за запуск против 0.42 у контроля — примерно в двенадцать раз чаще[live].

Блок состояния ограничивает *промпт*, но не ограничивает *объём информации, которого требует задача*, и разницу агент покрывает через цикл вызовов инструментов — именно там, куда не смотрит ни одна метрика размера промпта. Это и есть тезис статьи, только сформулированный как измерение, а не как довод.

8.2 Прочитано вручную, одна траектория

Одна траектория, изученная напрямую, а не через агрегат[live]: 72 вызова инструмента, из них `input.tex` прочитан 22 раза (двумя разными написаниями пути), второй файл — 19 раз, документ перекомпилирован пять раз и каждый раз выдаёт те же семь предупреждений на тех же номерах строк. Ходы 22–33 — это почти сплошь чередующиеся повторные чтения двух файлов, чьи байты ни разу не менялись.

Агрегат может произвести и сломанный метод подсчёта; здесь всё было наоборот — сначала прочитали вручную, а метрику написали потом, чтобы воспроизвести уже увиденное механически.

8.3 Что на самом деле лежало в состоянии

Смотрим на финальное состояние 251 запуска с одним состоянием, отдельно по исходу[live]:

Маркер в финальном состоянии	Решено	Провалено
Символов (медиана)	1218	963
Путей к файлам	9	5
Точных версий	1	**0**
Текста ошибки или кода завершения	0	**0**
Процитированных команд	1	**0**

В состоянии провалившегося запуска нет ни одной точной версии, ни одного текста ошибки, ни одной команды — то есть именно тех фактов, которые агент затем заново добывает, повторно запуская ту же команду. Схеме попросту некуда их записать: её поля — это прозаические контейнеры (`goal`, `facts_established`, `work_done`, `work_remaining`, `next_step`, `current_blocker`).

Связь идёт в ожидаемую сторону[live]: запуски, чьё состояние фиксирует не менее восьми путей к файлам, решаются в 0.50 случаев против 0.24 у тех, что фиксируют меньше; запуски с записанной точной версией — в 0.47 против 0.32; запуски с записанной ошибкой или кодом завершения — в 0.50 против 0.33. Это корреляция (запуск, продвинувшийся дальше, просто имел что записывать), но это единственное направление, куда указывают данные, и оно и мотивирует исследование композиции в §10.

8.4 Почему исходная оценка не могла этого увидеть

Повторное выяснение тратится дополнительными шагами по инициативе самого агента. Генератор оригинала фиксирует число шагов на эпизод (§6), так что механизму, измеренному в этом разделе, просто негде проявиться. Именно в этом смысле синтетический результат не ошибочен, а неприменим: бенчмарк убирает саму переменную, через которую идёт плата.

9 Результаты III: экономика

9.1 Стоимость при равном исходе

Сравнивать стоимость между руками, решающими разное число задач, бессмысленно: неуспешный запуск останавливается раньше и потому стоит дешевле. Каждое отношение в Таблице 5 вычислено только по парам, где *обе* руки решили одну и ту же задачу в одном и том же испытании, и каждое отношение — медиана по парам, так что задать число одной дорогой задаче не под силу.

При равном исходе опубликованная конфигурация несёт $0.47\times$ от пикового промпта контрольной группы и стоит $2.57\times$ дороже ($p = 1.4 \times 10^{-12}$). Промпт вдвое меньше, а счёт — в два с половиной раза больше.

Таблица 5: Отношения ресурсов против контрольной группы того же испытания, по парам, где обе руки решили задачу. Медианы по парам; p из точного знакового теста по отношениям стоимости.

Condition	Pairs	Cost	Peak prompt	Output	Cached	p
State-only (k=0)	78	2.57×	0.47×	4.55×	0.89×	1.4×10^{-12}
State +2	102	1.39×	0.60×	2.08×	0.40×	4.7×10^{-4}
State +6	131	1.15×	0.81×	1.36×	0.21×	0.04

Таблица 6: Итоги по серии. Приводятся только для масштаба: суммы по условию — нечестное сравнение, потому что неуспешный запуск останавливается раньше.

Condition	Cost	Turns	Input	Cached	Output
Control (full transcript)	$17.85	6,504	18.7M	352.2M	9.0M
State-only (k=0)	$14.83	16,072	27.3M	25.9M	19.9M
State +2	$12.88	10,250	27.6M	15.9M	16.5M
State +6	$11.83	4,725	38.9M	6.2M	11.6M

9.2 Куда уходят деньги

Та же таблица показывает механизм. Выходные токены составляют 4.55× от контрольной группы: блок состояния регенерируется на каждом ходу, а сгенерированный текст тарифицируется по самому дорогому тарифу. Кэшированные чтения убывают по мере того, как сохраняемая история сокращается, и итоги по серии показывают тот же эффект с другой стороны: контрольную группу по всей серии обслуживают 352.2M кэшированных токенов против 6.2M у state+6^{live}.

Только дополняемый транскрипт обслуживается кэшем почти целиком, потому что каждый вызов разделяет префикс с предыдущим. Блок состояния переписывают на каждом ходу, поэтому он расходится с предыдущим префиксом в точке, где начинается блок, и скидка, которая делает контрольную группу дешёвой, там уже не действует. Ни одна из этих величин не видна в совокупном подсчёте токенов — в этом и состоит конкретное содержание методологического утверждения: сокращение токенов и сокращение стоимости — разные измерения, и на этом корпусе у них противоположные знаки.

9.3 Решённые задачи на единицу затрат на уровне серии

Рантайм, вдвое сокращающий число токенов и проваливающий половину задач, — не улучшение. Взгляд с точки зрения эффективности (Таблица 7) делит исходы на ресурсы.

Отсюда следуют две вещи. На токен руки состояния действительно выглядят эффективными (1.24 против 0.42 решённых на миллион у контрольной группы) — это метрика оригинала, и именно такое прочтение даёт оценка по числу токенов. На доллар ранжирование переворачивается: state-only оказывается худшей из всех четырёх рук при 6.1 решённых на доллар — ниже контрольной группы с 9.0.

По этой мере на уровне серии лучшая рука — state+6 при 13.0 решённых на доллар против 9.0 у контроля. Читать это сравнение нужно с оговоркой из §9.1: неуспешный запуск останавливается раньше и стоит немного, так что итоги по серии приукрашивают ту руку, которая проваливает больше задач, и здесь они приведены только для масштаба, не как вердикт.

Дело решает сравнение при совпадающем исходе. По нему state+6 *не* дешевле контрольной группы: 1.15× по 131 задачам, решённым обеими руками, — дороже в 78 из них против 53 ($p = 0.04$)^{live}. Честное утверждение об экономике поэтому уже, чем предполагает подсчёт токенов в любую сторону. Восстановление короткого хвоста устраняет дефицит способности решать задачи (§7) и снижает премию по стоимости с 2.57× до 1.15×, но на этом корпусе ни одна конфигурация

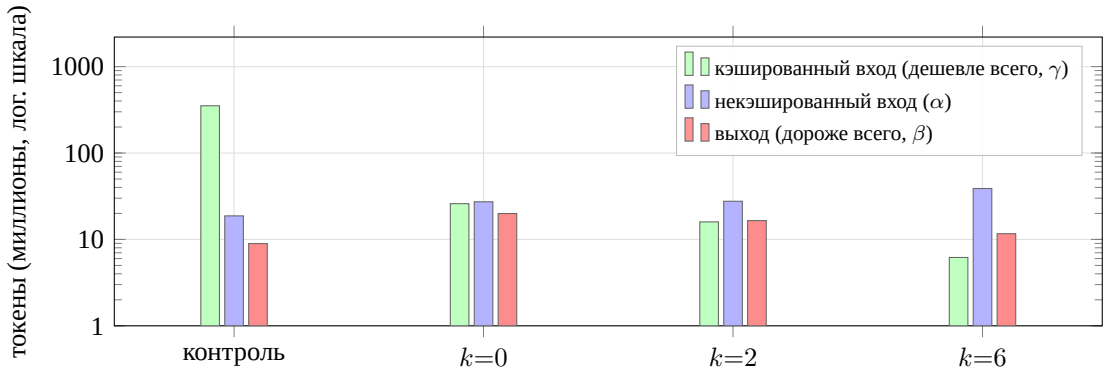


Рис. 3: Потребление токенов по тарифицируемым классам по всей серии, декомпозиция, которую тарифицирует Ур. (8). Столбцы сгруппированы, а не составлены друг на друга, а ось логарифмическая, потому что классы различаются на два порядка. Контрольная группа потребляет абсолютно больше всего токенов, но 352.2M из них — кэшированные чтения, тарифицируемые по самой дешёвой ставке: только дополняемый промпт почти полностью обслуживается из кэша префикса. Руки состояния потребляют намного меньше токенов и платят за них больше, потому что блок состояния, переписываемый на каждом ходу, теряет этот кэш (вплоть до 6.2M) и потому что поддержание состояния переносит токены в класс выхода. Именно поэтому сравнение по числу токенов и сравнение по стоимости расходятся.

Таблица 7: Эффективность. Решённых задач на миллион токенов и на доллар по всей серии.

Condition	Solved	Tokens	Cost	Solved/Mtok	Solved/$
Control (full transcript)	160	379.9M	$17.85	0.42	9.0
State-only (k=0)	91	73.1M	$14.83	1.24	6.1
State +2	118	60.1M	$12.88	1.96	9.2
State +6	154	56.7M	$11.83	2.72	13.0

структурированного состояния не вышла дешевле простого сохранения транскрипта при равном исходе.

9.4 H2 опровергнута согласно предрегистрации

Предрегистрация предсказывала, что отношение стоимости упадёт с ростом горизонта и пересечёт 1.0 — то есть что накладные расходы амортизируются на длинных задачах. Этого не происходит: корреляция Спирмена между горизонтом контрольной группы и отношением стоимости по задаче составляет +0.16 по 35 задачам ($p = 0.36$) — неотличима от нуля и, если уж на то пошло, не того знака.

Тест охватывает 35 из 87 задач по самому определению этой величины, а не по какому-то правилу исключения: отношение стоимости по задаче существует только там, где и контрольная группа, и state-only решили задачу хотя бы в одном испытании, а state-only решает лишь 0.349 запусков. Это ограничение следует из результата о способности решать задачи в §7, и оно играет против нас: задачи, на которых state-only провалилась полностью, — это как раз те длинные задачи, на которых предсказывалась амортизация.

В наблюдаемом диапазоне горизонта пересечения нет. Мы сообщаем об этом как об опровержении собственной гипотезы, а не отбрасываем её молча, и это согласуется с §7: стоимость не только не амортизируется — дефицит способности решать задачи с горизонтом ещё и расширяется.

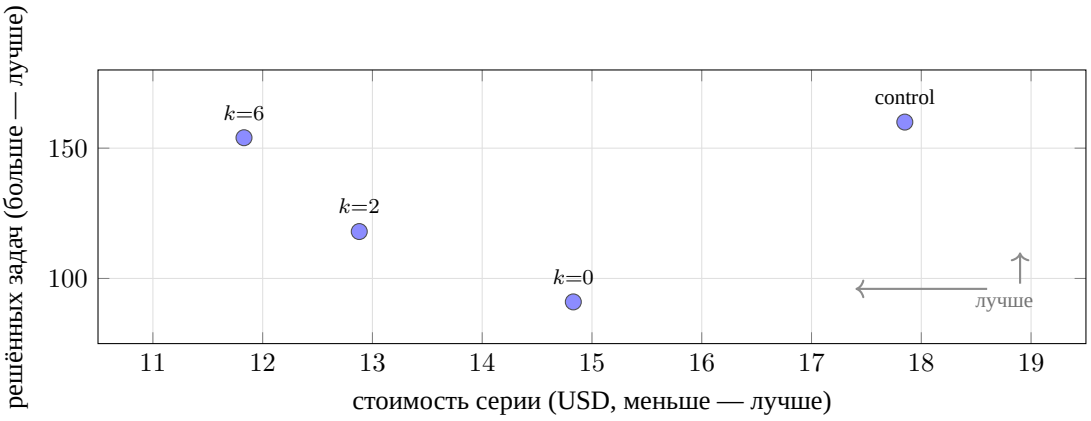


Рис. 4: Граница, которая имеет значение на практике: решённые задачи против потраченных денег, вся серия. Опубликованная конфигурация ($k{=}0$) доминируема: она тратит меньше контрольной группы, но решает намного меньше задач, и обе $k{=}2$ и $k{=}6$ превосходят её по обеим осям одновременно. Читайте этот рисунок только как обобщение на уровне серии. Поскольку неуспешный запуск останавливается раньше и потому стоит немного, итоги по серии приукрашивают руки, которые проваливаются чаще, поэтому именно сравнение при совпадающем исходе из Таблицы 5, а не этот рисунок, несёт утверждение статьи о стоимости. По этому честному сравнению $k{=}6$ *не* дешевле контрольной группы (1.15×, $p = 0.04$). Что рисунок действительно показывает — это то, что ни один из крайних случаев оси хвоста не является правильным ответом.

10 Композиция состояния: кто пишет состояние

Разделы 7–9 устанавливают, что ущерб происходит от отброшенной истории, а платит за него агент, заново добывая информацию через свои инструменты. Из этого диагноза следует и лечение: если бы состояние уже несло факты, которые агент постоянно выводит заново, повторное приобретение стало бы не нужно.

Этот раздел варьирует одну вещь: сколько состояния пишет *рантайм*, не спрашивая модель. Это другая ось, отличная от хвоста, и рассматривается она отдельно, потому что отвечает на другой вопрос. Ось хвоста спрашивает, сколько истории хранить. Эта ось спрашивает: при фиксированном объёме истории (никакой) — кто должен производить состояние, которое её заменяет.

10.1 Фактор

В skill.state блок состояния — сгенерированный текст: модель выдаёт $\Delta\Sigma_t$ в той же генерации, что и своё действие (Алгоритм 2). Из §9 следуют два неблагоприятных следствия. Поддержание состояния тарифицируется по выходному тарифу β — самому дорогому классу в Ур. (8). И модель вольна переписать факт, который сама же записала раньше; собственная таксономия ошибок оригинала приписывает этому 68 % отказов моделей с открытыми весами^{orig}.

Альтернатива — позволить рантайму фиксировать то, что уже произошло. Результат работы инструмента — пронаблюдённый факт, а не утверждение: рантайм видел, какие файлы были затронуты, какая команда выполнена, что она вывела и прошли ли тесты. Записать это в состояние не требует генерации и не может быть переписано моделью.

Фактор — сколько именно записывать. Лестница добавляет один компонент на ступень:

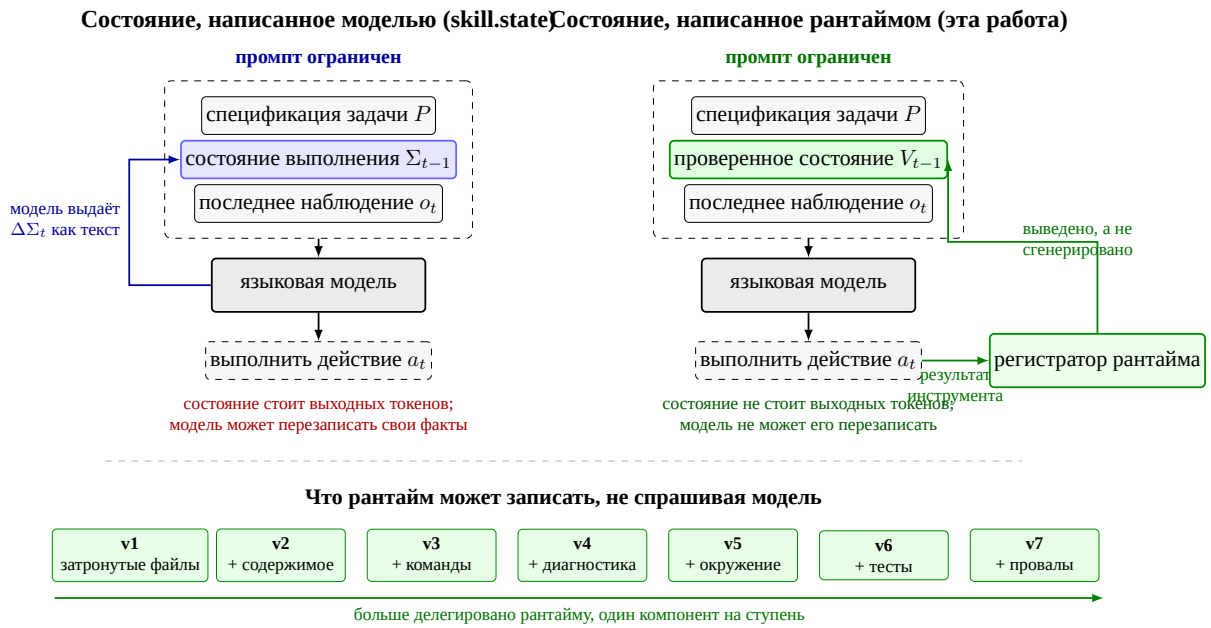


Рис. 5: Два способа ограничить промпт блоком состояния, различающихся тем, кто его пишет. **Слева:** модель выдаёт обновление состояния как сгенерированный текст, так что его поддержание тарифицируется по выходному тарифу, а модель вольна перезаписать ранее записанный факт, чему оригинал приписывает 68 % своих отказов моделей с открытыми весами[orig]. **Справа:** рантайм выводит ту же запись из результатов инструментов, которые уже произошли. Блок — это факт о том, что случилось, а не утверждение модели об этом, он не стоит сгенерированных токенов и не может быть перезаписан моделью. Полоса внизу — это фактор, варьируемый в §10: каждая ступень передаёт рантайму ещё один компонент, при замороженных хвосте и схеме, так что разрыв между соседями называет один делегированный компонент.

Ступень	Добавляет	Что пишет рантайм
v1	файлы	пути, к которым обращался агент
v2	+ содержимое	записи изменений по файлам и глобальные
v3	+ команды	нормализованные записи команд с их выводом
v4	+ диагностика	структурированный вывод компилятора и линтера
v5	+ окружение	версии, пути, состояние интерпретатора
v6	+ тесты	вызовы тестов и их результаты
v7	+ провалы	записи о том, что пробовали и что не сработало

Хвост заморожен при $k{=}0$, а схема — на `L2-semantic` в каждой ячейке, так что ни одна ступень не заимствует преимущество у факторов из §7. Соседние ступени различаются ровно одним компонентом, и этот инвариант утверждается по флагам, которые условия фактически выдают перед запуском любой ячейки, а не по замыслу, стоящему за ними[code].

Контрольная группа выполняется внутри этой серии. Мы пробовали повторно использовать контрольную группу E1, и данные это отвергли: та же рука на тех же 87 задачах набрала 0.793 в E1 и 0.552 здесь, разойдясь по 31 из них[live]. Стенд дрейфует между сериями, так что заимствованная контрольная группа измеряет не только условие, но и этот дрейф. Против заимствованной контрольной группы первая ступень выглядела дефицитом 46–2; против контрольной группы внутри серии — 25–2. Отсюда правило: одна контрольная группа на серию.

Таблица 8: Лестница композиции. Стоимость, ходы и вывод — медианы по задачам, которые решили и ступень, и контрольная группа; отношения ниже 1 благоприятствуют ступени.

Rung	Adds	Solved	Rate	Cost ratio	Turns	Output
control	full transcript	48	0.552	$1.00\times$	$1.00\times$	$1.00\times$
v1	files	25	0.287	$2.31\times$	$1.62\times$	$3.96\times$
v3	+ commands	35	0.402	$1.20\times$	$0.79\times$	$1.64\times$
v6	+ tests	37	0.425	$0.92\times$	$0.71\times$	$1.84\times$
v7	+ failures	36	0.414	$0.94\times$	$0.73\times$	$1.73\times$

Таблица 9: Парные шаги вдоль лестницы, одна и та же задача и испытание.

Step	Lower solves only	Higher solves only	p
v1 $\rightarrow$ v3	2	12	0.01
v3 $\rightarrow$ v6	9	11	0.82
v6 $\rightarrow$ v7	5	4	1.00
v1 $\rightarrow$ v7	3	14	0.01

10.2 Способность решать задачи улучшается по мере делегирования

Таблица 8 приводит лестницу. Первая ступень — состояние, хранящее только список затронутых файлов, — решает 0.287 против 0.552 у контрольной группы. Добавление записей команд поднимает её до 0.402, тесты — до 0.425. На последней ступени подъём не продолжается: добавление записей о провалах даёт 0.414— на одну задачу меньше, чем v6, и при 5 против 4 несовпадающих задач это неотличимо от отсутствия изменений. Лестница, таким образом, не монотонна на своей вершине, хотя это падение лежит в полосе шума, а не за порогом парного сравнения.

Парные ступени приведены в Таблице 9. Сам по себе значим только первый шаг: переход от файлов к командам выигрывает 12 задач и проигрывает 2 ($p = 0.01$). Более поздние отдельные шаги при таком размере выборки по отдельности неотличимы от шума (9 против 11, $p = 0.82$; и 5 против 4, $p = 1.00$). А вот сквозной шаг значим: переход от v1 к v7 выигрывает 14 и проигрывает 3 ($p = 0.01$).

Поэтому мы утверждаем тренд, а не отдельные ступени. Данные подтверждают, что состояние, несущее больше выведенных рантаймом фактов, решает больше задач, чем состояние, несущее меньше их, причём основная часть выигрыша приходит рано — в точке, где команды и их вывод входят в запись. Чтобы локализовать оставшиеся компоненты друг относительно друга, нужны испытания, которые всё ещё выполняются.

10.3 Стоимость падает в то же время

Результат, который имеет значение, — в последних трёх столбцах Таблицы 8: способность решать задачи и стоимость движутся вместе, а не друг против друга.

На v1 рука стоит $2.31\times$ от контрольной группы при равном исходе. Это тот самый режим из §9: состояние, почти ничего не записывающее, вынуждает агента заново добывать почти всё, а на это уходит $1.62\times$ от числа ходов контрольной группы. К v6 то же сравнение даёт $0.92\times$ — уже ниже контрольной группы, а ходы упали до $0.71\times$. Рука, делегирующая больше всего, одновременно и наиболее способна на этой оси, и наиболее дешёва.

Механизм виден в столбце вывода. На v1 рука генерирует $3.96\times$ от выходных токенов контрольной группы; к v6 это уже $1.84\times$. За факты, которые пишет рантайм, не платят по выходному тарифу, так что перенос компонента от модели к рантайму убирает сгенерированные токены и одновременно устраняет то самое повторное приобретение, которое эти токены всё равно не могли

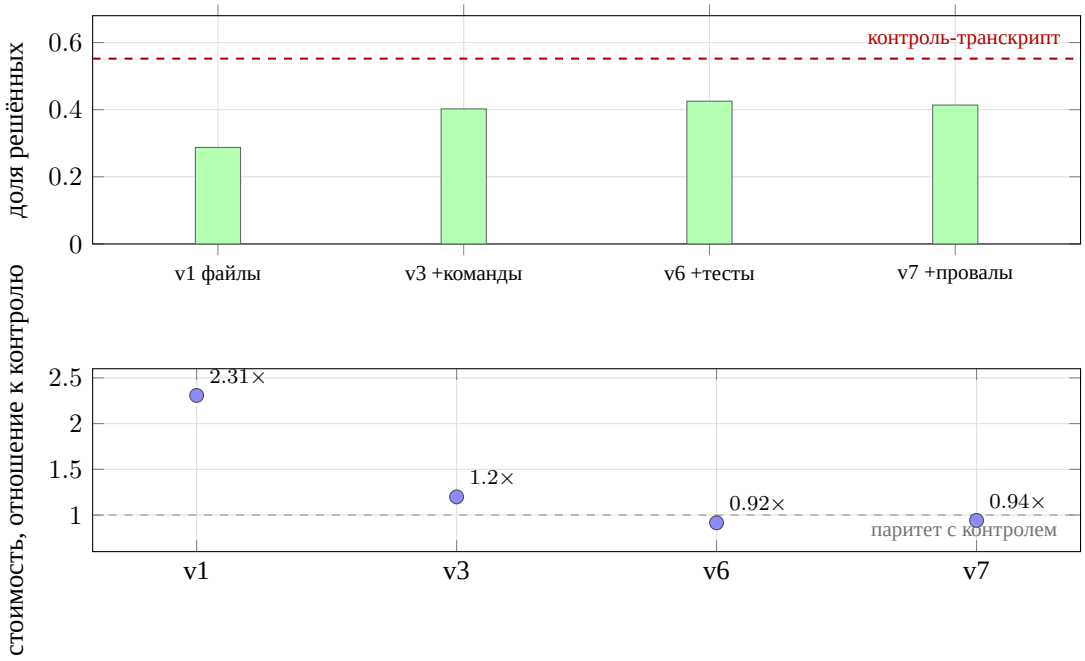


Рис. 6: Делегирование большей части состояния рантайму двигает способность решать задачи и стоимость в одном направлении, что противоположно обычному компромиссу. **Сверху:** доля решённых задач по ступени против контрольной группы транскрипта внутри серии (пунктир). Первая ступень восстанавливает 0.287 задач; передача команд, тестов и записей о провалах достигает 0.425, закрывая примерно половину дистанции до контрольной группы без восстановления какого-либо транскрипта. **Снизу:** медианная стоимость на задачах, которые решили и ступень, и контрольная группа. Те же добавления, что поднимают способность решать задачи, также снижают стоимость с 2.31× контрольной группы до 0.92×, ниже паритета. Состояние, выведенное рантаймом, дешевле состояния, написанного моделью, потому что оно никогда не генерируется по выходному тарифу.

Таблица 10: Каждая ступень в паре с контрольной группой транскрипта внутри серии.

Rung	Control only	Rung only	p
v1	25	2	5.6×10^{-6}
v3	18	5	0.01
v6	17	6	0.03
v7	16	4	0.01

предотвратить.

На доллар по всей серии порядок тот же: 12.9 решённых задач на доллар на v1 против 20.9 на v6, при контрольной группе на 22.4.

10.4 Чего это не показывает

Лестница не дотягивается до контрольной группы. Каждая ступень остаётся позади неё по парной способности решать задачи (Таблица 10), и даже лучшая всё ещё в минусе на -11 задач против агента с транскриптом. Делегирование большего рантайму — строгое улучшение *над состоянием, написанным моделью*, но на этом корпусе оно ещё не заменяет сохранение недавней истории.

В сочетании с §7, где всего шесть дословных обменов полностью закрывают разрыв, практический вывод таков: два механизма дополняют друг друга, а не соревнуются. Состояние, написанное рантаймом, — дешёвый способ нести устойчивые факты, а короткий хвост — дешёвый способ нести недавний контекст. Здесь ни один из них не заменяет другой.

Статус этих чисел. 435 запусков, 87 задач, одно испытание. E1 несёт три испытания; эта ось — одно, а в этом проекте уже случался эффект, значимый при двух испытаниях и умерший на третьем (§12). Направление согласуется на каждой ступени, и сквозное сравнение значимо, но отдельные шаги от ступени к ступени следует читать как предварительные, пока не поступят оставшиеся испытания. Ни одно утверждение в этом разделе не опирается на собственные счётчики инструментирования плагина: они тождественно нулевые по всей серии, и мы трактуем это как мёртвое инструментирование, а не как свидетельство отсутствия[live].

11 Угрозы валидности

Одно семейство моделей. Каждый запуск использует `zai/glm-5.3-flash`. Парный пилот на более сильной модели того же провайдера не решил ни одной задачи сверх того, что решала маленькая модель, — при 22.7× медианной стоимости на ячейку[live], и именно поэтому серию не стали переносить на неё. Этот пилот — всего 7–8 задач, он закрывает вопрос бюджета, а не вопрос переносимости. Мы ничего не утверждаем о моделях других семейств и, в частности, о том, поддерживала бы модель с более сильным следованием инструкциям состояние настолько хорошо, чтобы избежать режима отказа из §8.

Один корпус и его диапазон горизонта. Задачи Terminal-Bench 2.0 при контрольной группе используют медианно лишь около 5 % окна контекста[live]. Окно поэтому редко становится связывающим ограничением — а это как раз тот режим, где механизм памяти должен быть наименее нужен. На более длинном Terminal-Bench 4.0 пилот показывает медианный пик уже в 66 % окна[live], и там тот же механизм проявляется на порядок сильнее (медианное повторение 3.26 против 1.02 у контрольной группы). Этот пилот — одно испытание на шести задачах, результатом его считать нельзя. Честно про охват можно сказать так: наша находка держится там, где окно не насыщено, а направление эффекта в насыщенном режиме измерено, но не установлено.

Точность реализации. Наш блок состояния на шаге толще, чем сообщает оригинал[live], так что абсолютные величины стоимости — верхние границы (§6). Обе поправки работают против нашего ключевого результата.

Перенос тарифов в воспроизведении. Раздел 6 применяет наши измеренные цены к опубликованным числам токенов оригинала. Порядок величины от точных множителей не зависит; они — оценка, и помечены как таковая.

Контаминация, принципиально не обнаруживаемая. Правило карантина (§12) опирается на ошибочные ходы. Деградацию провайдера ниже порога видимости ошибок оно бы не поймало. Границу ущерба задают два факта: результаты по стоимости вообще не зависят от вердиктов, а результат по способности решать задачи пережил удаление каждой ячейки, которую правило всё же поймало.

Две невыполнимые задачи. Две из 89 задач отказывают при настройке контейнера при каждом условии[live] и отсутствуют во всех руках одинаково. Это убирает две задачи из корпуса, но не даёт преимущества какому-либо условию.

Исследование композиции несёт одно испытание. Раздел 10 приводит 435 запусков за одно испытание против трёх для E1. С учётом собственной истории проекта (эффект, значимый при двух испытаниях, умер на третьем (§12)), отдельные шаги от ступени к ступени там нужно считать предварительными. Сквозное сравнение и направление тренда стоимости согласуются на каждой

ступени — поэтому они вообще приводятся, — но ни одно утверждение в аннотации или заключении этой статьи на них не опирается.

Нет руки только-хвоста. Четыре условия E1 — это контрольная группа без блока состояния и три руки, несущие состояние с разными хвостами. Руки, которая несла бы последние k обменов и *не* несла бы блока состояния, среди них нет. Без неё §7 может показать, что восстановление истории устраняет дефицит, но не может отделить, какая часть оставшегося поведения при $k{=}6$ приходит от блока состояния, а какая — от истории рядом с ним. Таблица 6 подсказывает, что это важно: state$+6$ завершает серию за меньшее общее число ходов, чем контрольная группа, а чисто аддитивный учёт двух компонентов такого исхода очевидным образом не предсказывает. Мы отмечаем это как необъяснённое, не интерпретируя, и этой руке место в следующей серии.

12 Подводные камни измерения и два отзыва

У оценок памяти агентов есть характерный режим отказа: числа реальны, скрипт верен, а вот метка на столбце — нет. Каждый пункт ниже стоил этому проекту целой серии, прежде чем стать правилом, и приводится он потому, что правило переносится даже там, где наши результаты — нет.

12.1 Пять подводных камней

1. **Самоотчёт модели — не вердикт.** Решает только собственный верификатор задачи. Каждая альтернатива, которую мы пробовали, производила строки, похожие на данные, но данными не бывшие.
2. **Экспериментальная рука может незаметно стать контрольной группой.** Если плагин не загрузился, рука продолжает работать как базовая линия под другим именем, а её числа остаются правдоподобными. Поэтому проверка сверяет реализованный состав контекста с промптом, обращённым к провайдеру, а не с собственным логом плагина[code].
3. **Детектор может совпадать с конфигурацией вместо событий.** Детектор тайм-аутов в этом проекте совпал со статической строкой политики повторов, печатаемой в каждом логе, и сообщил о 54 тайм-аутах там, где их не было[live]. Дефект проявил себя лишь через невозможное нижестоящее число: долю решённых задач выше 100 %. Его модульный тест закодировал ту же ошибку, поэтому набор тестов был зелёным — код и тест ошибались совершенно одинаково. Отсюда принятое правило: прежде чем подпустить детектор к корпусу, его проверяют на запуске с заранее известной истиной.
4. **Насыщение провайдера читается как отказ метода.** При конкурентности выше измеренного потолка провайдера 68 % запросов были ограничены по скорости[live]. Изголодавшийся запуск всё равно получает от своего верификатора честный ноль, и этот ноль попадает в таблицу как «метод провалился». Контаминация однонаправленна по построению: контрольная группа уже успела завершиться в чистых условиях, так что ходы теряли только экспериментальные руки. Правило карантина зафиксировали ещё до того, как увидели подсчёты: чистым считается ноль ошибочных ходов, номинальным — не более двух и меньше 5 %, всё остальное исключается.
5. **Недостаточная мощность бинарных конечных точек.** Перед этой серией мы вычислили, что для обнаружения различия в доле решённых задач нужно около 72 задач при измеренной частоте несовпадений, и заявили об этом заранее, вместо того чтобы задним числом выдать нуль за эквивалентность. Текущая серия это требование покрывает — именно поэтому §7 вообще может утверждать что-то о способности решать задачи.

12.2 Два отзыва

Оба приводятся, потому что отзыв — тоже результат об измерении.

«Цензурирование по лимиту времени объясняет разрыв в способности»: отозвано, цензурирования никогда не существовало. Это было следствие из дефекта детектора, описанного в пункте 3. После исправления детектора истинное число цензурированных по времени запусков в корпусе оказалось нулевым. Последующее наблюдение о том, что повышение лимита вернуло четыре из семи провалившихся задач, было затем проверено против измеренной базовой частоты восстановления вердикта между неизменёнными повторными запусками (5/11); вероятность увидеть четыре или более восстановлений при одной этой базовой частоте — 0.40^{live}. Ничем не примечательно, отозвано.

«Только-состояние менее стабильно»: отозвано на третьем испытании. После двух испытаний рука только-состояния показывала более высокую дисперсию между испытаниями на 10 из 12 задач ($p = 0.039$). Третье испытание сдвинуло это к 8 из 12 ($p = 0.388$). Утверждение умерло ровно так, как и следовало ожидать от двухиспытательного p чуть ниже 0.05. Остановись серия на двух испытаниях, в статью попал бы ложный эффект.

Чего отзывы не затронули. Каждое измерение стоимости, токенов и кэша в этой статье не зависит от вердиктов, а результаты по способности решать задачи из §7 получены дизайном, появившимся позже всего перечисленного выше и покрывающим требование к мощности, которое его и мотивировало.

12.3 Промежуточный случай контаминации, сохранённый как предупреждение

Ранний анализ схемы в этом проекте дал $p = 7.9 \times 10^{-8}$ в пользу сильного эффекта. При повторном запуске того же сравнения по тому же скрипту на карантинно-очищенном корпусе получилось $p = 9.2 \times 10^{-2^{\mathsf{live}}}$. Контаминированные ячейки были сконцентрированы именно в сравниваемых руках: арифметика была верна, а с толку сбивала именно шапка столбца.

13 Связанные работы

Память со структурированным состоянием. skill.state [Badhe et al., 2026] — проверяемая система, описанная в §2. Всё семейство, к которому она принадлежит (замена истории поддерживаемым моделью структурированным представлением), разделяет предположение, которое мы фальсифицируем: что представление — достаточная статистика для оставшейся работы.

Детерминированное состояние выполнения. ledger [Wang et al., 2026] поддерживает объективную запись, выводимую из завершённых взаимодействий, и добавляет её к иначе немодифицированному агенту, который сохраняет свою историю. Наше исследование композиции (§10) варьирует границу между этими двумя дизайнами, и стоит сказать прямо, в чём разница условий: ledger *добавляет* детерминированный слой агенту с полной историей, а мы спрашиваем, что должен содержать детерминированный слой, когда история уже *удалена*.

Управление контекстом через сжатие. Рантаймы суммаризации, скользящие окна и обученное сжатие промпта [Jiang et al., 2023] решают ту же проблему роста иначе — сжимая транскрипт вместо того, чтобы его заменять. Контрольные группы оригинала с подобранным бюджетом показывают, что при равном размере промпта они набирают заметно меньше, чем структурированное состояние^{orig}— это свидетельство в пользу того, что выигрыш даёт структура, а не краткость, и

наши результаты этому выводу не противоречат. Наша находка — о том, что теряется при полном отбрасывании транскрипта, а не о том, насколько хорошо его можно сжать.

Компоненты харнеса по отдельности. Fan et al. [2026] прогоняют 176 сопоставленных настроек, независимо варьируя планирование, пространство действий и управление контекстом внутри одного фиксированного цикла выполнения, на четырёх моделях на SWE-Bench Verified и Terminal-Bench 2.1. Их условие контекста с исключением-и-вызовом — ближайший опубликованный аналог нашей оси хвоста (§7): оба урезают транскрипт и дают агенту механизм вернуть то, что было урезано. По их данным, канал вызова добавляет мало, как только исключение уже установлено — и это зеркалит нашу собственную находку: счётчики покрытия `verified_*` у плагина тождественно нулевые в каждом запуске серии композиции (§10) — канал, построенный специально, чтобы проверить, что ничто из 435 запусков его не задействует. Мы читаем это как один и тот же режим отказа, до которого добрались из двух разных дизайнов, а не как совпадение названий.

Слепота к контексту. Xu et al. [2026] утверждают, что модель не способна надёжно вывести размер блока, недавность или оставшийся бюджет только из промпта, и вводят видимый канал внутреннего состояния, чтобы агент видел, чем он управляет. Наш результат о механизме (§8) с этим диагнозом согласуется, хотя к нему прямо и не обращается: неуспешный запуск только-состояния тратит большинство своих команд на то, чтобы заново вывести уже произведённые факты — именно это и предсказывает слепота к контексту для агента, недостаточно осведомлённого о собственной истории. Их исправление работает с тем, как модель видит свой контекст; наше (§10) — с тем, кто изначально пишет запись. Друг с другом они не конкурируют, и §14 называет их сочетание направлением, которое мы не измеряли.

Не теряющий данных рекурсивный контекст. Ehrlich and Blackman [2026] поддерживают детерминированное дерево иерархических сводок с не теряющими данных указателями обратно к исходному тексту, позволяя агенту рекурсивно углубляться в узел сводки, не теряя возможности извлечь то, что было сведено. По сути это то же базовое утверждение, что и у ledger в нашей постановке — детерминированный слой способен нести информацию, которую собственная свободная сводка модели отбросила бы, — только применённое рекурсивно, а не как один дополнительный слой. Отсюда следует, что граница, которую мы изучаем в §10 — написанное моделью против написанного рантаймом состояние, — не единственная доступная ось: детерминированный слой может варьироваться и по глубине.

Агентные циклы рассуждения. Базовая линия транскрипта устроена в стиле ReAct [Yao et al., 2023]: чередующиеся рассуждение и действие при только дополняемой истории. Именно её профиль стоимости при кэшировании префикса делает сравнение в §9 неочевидным, поскольку растущий промпт в основном обслуживается кэшем.

Бенчмарки с программными верификаторами. Мы используем Terminal-Bench, чьи задачи несут собственные верификаторы, по причине, названной в §5: вердикт не должен зависеть от того, как модель судит о собственном выводе. InterCode CTF [Yang et al., 2023] и τ-bench [Yao et al., 2024], которые использует оригинал, разделяют это же свойство.

Методология оценки. Наш методологический вклад (§12) переносит стандартную практику воспроизводимости (внешние вердикты, предрегистрированная мощность, письменные отзывы) в жанр, где, по нашему опыту, доминирующая ошибка носит не статистический, а определительный характер: столбец, помеченный как свойство метода, на самом деле измеряет харнес.

14 Заключение

Память со структурированным состоянием мотивирована верным наблюдением: неограниченный транскрипт — плохая рабочая память, и его стоимость растёт со всем, что было прежде. Опубликованная оценка skill.state показывает, что поддерживаемый моделью блок состояния удерживает рост промпта почти на постоянном уровне, не теряя в качестве, и на собственном бенчмарке этот результат воспроизводится — в том числе при учёте кэша.

На реальных программных задачах он не держится. По 1044 запускам предрегистрированного дизайна метод в опубликованном виде решает 0.349 задач против 0.613 у контрольной группы с транскриптом, проигрывая 82 парных задач и выигрывая лишь 13 ($p = 2.1 \times 10^{-13}$), причём дефицит *расширяется* именно на длинных горизонтах, где механизм должен был окупать себя. Восстановление шести дословных обменов рядом с блоком состояния полностью устраняет разницу — а значит, ущерб коренится в отброшенной истории, а не в самой абстракции состояния.

Причина, по которой две постановки расходятся, — структурная, а не эмпирическая. Синтетический бенчмарк фиксирует число обращений к модели за эпизод, поэтому агенту, отбросившему что-то нужное на будущее, попросту нечем платить за восстановление этого. Реальные агенты платят: в неуспешных запусках только-состояния 58 % всех команд — это повтор уже выполненного осмотра, против 20 % у контрольной группы. Вот тезис статьи в одном предложении: *ограничение промпта не ограничивает информацию, которую требует задача; разницу оплачивают через цикл инструментов, куда не заглядывает ни одна метрика размера промпта.*

Экономика следует из того же механизма и переворачивает привычное прочтение. При равном исходе рука только-состояния несёт половину пикового промпта контрольной группы, но стоит 2.57× дороже: поддержание состояния тарифицируется по выходному тарифу, а блок, переписываемый на каждом ходу, теряет кэш префикса, который и делает дополняемый транскрипт дешёвым. Если мерить на токен, руки состояния выглядят эффективными; если мерить на решённую задачу на доллар по всей серии, опубликованная конфигурация оказывается худшей из четырёх. Но на сравнении при совпадающем исходе, которое эта статья считает честным, дороже контрольной группы обходится каждая конфигурация состояния, включая лучшую (1.15× при k=6, $p = 0.04$).

Практический вывод поэтому не в том, что от структурированного состояния нужно отказаться, а в том, что им не следует *заменять* недавнюю историю. Конфигурация, выигрывающая на этом корпусе, сохраняет и то, и другое, и открытый вопрос, который мы измеряем дальше, — что должно содержать поддерживаемое рантаймом состояние, чтобы оставшийся разрыв закрылся (§10).

Лестница композиции из §10 останавливается на v7 (записи о том, что пробовали и что не сработало) просто потому, что там остановились наши собственные запуски — не потому, что там заканчивается пространство дизайна. Два механизма, упомянутые в §13, находятся на шаг дальше, и ни один из них на этом корпусе не запускался. Канал проприоцепции VISTA [Xu et al., 2026] дал бы агенту явный сигнал о том, что уже содержится в написанном рантаймом состоянии, а это напрямую связано с повторным выводом, который мы измеряем в §8: блок состояния, форму которого агент не видит, — это блок состояния, который он будет перепроверять. Рекурсивные, не теряющие данных указатели LCM [Ehrlich and Blackman, 2026] позволили бы ступени суммировать собственную историю вместо того, чтобы только накапливать её, — это решило бы тот единственный режим отказа, который лестница не может исправить добавлением новых полей: неограниченный рост самого блока состояния на высоких ступенях. Ступень, сочетающая оба подхода — написанное рантаймом состояние, форму которого агент может запросить, рекурсивно суммированное без потери возможности извлечь то, что было сведено, — естественная v8 и самая прямая проверка того, продолжается ли подъём лестницы или он уже вышел на плато к v6. Мы её не строили и не запускали; здесь она названа как следующее измерение, а не заявлена как результат.

Методологический вывод, возможно, переживёт эмпирический. В этом жанре различие между находкой и артефактом редко упирается в модель, метод или бенчмарк. Оно упирается в учёт: в

какой валюте измеряется стоимость, какую руку штрафует ограничение харнеса и по-прежнему ли шапка столбца описывает то, что в столбце на самом деле содержится.

Список литературы

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. v3, 02 Sep 2026.

Clint Ehrlich and Theodore Blackman. LCM: Lossless context management. *arXiv preprint arXiv:2605.04050*, 2026.

Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.

Zehao Wang, Yisen Xu, Chenglin Li, Chao Peng, Bram Adams, Ahmed E. Hassan, and Tse-Hsun Chen. Turning interaction history into execution state: A runtime layer for long-horizon coding agents. *arXiv preprint arXiv:2608.00808*, 2026.

Binyan Xu, Haitao Li, and Kehuan Zhang. LLM agents are latent context managers: Eliciting self-managed context via state proprioception. *arXiv preprint arXiv:2606.30005*, 2026.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Воспроизводимость

Набор данных. 1044 запусков, одна строка на запуск, и по каждой строке — происхождение: идентификатор запуска, временные метки начала и завершения, ревизия репозитория, хэш содержимого плагина, хэш схемы, модель, испытание, условие, реализованное значение k, вердикт верификатора, причина остановки, число ходов и вызовов, три числа токенов по отдельности, пиковый промпт и производная стоимость. Каждую строку перекрёстно проверили поле за полем против сырого вывода сборщика — с нулевыми расхождениями[live].

Регенерация чисел статьи. Каждую таблицу, каждую координату рисунка и каждое встроенное число в этой статье производит один детерминированный анализ из опубликованных данных запусков. Ни одно измеренное число не вписано в исходник рукописи вручную: текст ссылается на сгенерированные определения, поэтому изменение данных не может оставить в прозе устаревшее число.

Анализ дополнительно проходит через гейт целостности, записанный как Алгоритм 3: он пересчитывает опубликованные ключевые значения из сырых строк и отказывается выдать документ,

Algorithm 3 Гейт целостности на этапе сборки

Require: каноническая таблица запусков D; опубликованные ключевые значения V
пересчитать каждую таблицу и встроенную величину из D
$M \leftarrow$ пересчитанные ключевые значения
for каждого $v \in V$ **do**
    if $M[v] \neq V[v]$ **then**
        сообщить о v с обоими значениями
        прервать сборку
    end if
end for
выдать таблицы, данные рисунков и определения макросов
типографировать документ

если хотя бы одно из них не совпало. Гейт проверили по построению: если исказить выборку вердиктов на входе, он падает и указывает каждую затронутую величину[live].

Точный вызов приведён в инструкциях по сборке репозитория, а не здесь.

Статистика. Точные тесты Макнемара и знаковый тест вычисляются из биномиального распределения в замкнутой форме; для долей используются интервалы Уилсона; тренд по горизонту оценивается ранговой корреляцией Спирмена с перестановочным p-значением при фиксированном сиде. Реализация опирается только на стандартную библиотеку и потому без изменений выполняется на измерительном хосте.

Исключения, с подсчётами. Попытки, не вернувшие вердикт верификатора, записываются отдельно и исключаются: подавляющее большинство из них позже переставили в очередь и закрыли с реальным вердиктом, а оставшееся распределено симметрично по рукам. Запуски, измеренные в условиях насыщения провайдера, карантинируются по заранее объявленному правилу из §12. Там, где перезапуск оркестратора выполнил одну ячейку дважды, билдер сохраняет самый ранний запуск — это правило зафиксировали ещё до того, как прочитали хоть один вердикт, чтобы повтор не мог протолкнуть себя в выборку победой[code].

Стек. Версии агента, харнеса и плагина зафиксированы и записываются для каждого запуска; хэши содержимого для плагина и схемы пишутся в каждую строку, так что любой запуск можно привязать к точному коду, который его произвёл.

Артефакты. Набор данных, сырой вывод сборщика, анализ и исходник рукописи публикуются вместе. [**NOTE:** DOI Zenodo будет присвоен при подаче; URL репозитория будет заполнен, когда релиз станет публичным]

B Пробелы отчётности в исходной оценке

Собрано во время полного извлечения текста исходной статьи, каждый пункт можно проверить по ней напрямую[orig]. Они записаны здесь потому, что несколько из них — причина, по которой воспроизведению приходится делать собственный выбор, а читатель должен знать, какой из этих выборов был вынужденным.

1. **Схемы.** Дословный список полей дан только для одного из четырёх доменов оценки. Для двух других схему можно вывести лишь из единственной примерной траектории, а для четвёртого не дано вообще ничего. Метод проектирования схемы нигде не описан, анализа чувствительности тоже нет. Воспроизводящему поэтому приходится изобретать схему самому,

что само по себе источник расхождения — и мотивация трактовать проектирование схемы как фактор, а не как константу.

2. **Кэширование и деньги.** Полнотекстовый поиск не находит ни одного вхождения слов cache, billing или иной денежной терминологии. Все экономические утверждения даны в логических числах токенов. Статья не кратка на этот счёт — она вообще молчит по этому вопросу.

3. **Дисперсия и повторения.** Таблицы устойчивости к шуму и восстановления состояния приводят точечные значения без указания стандартного отклонения или числа повторений.

4. **Контроль сжатия недоопределён.** Базовую линию обученного сжатия приводят без коэффициента сжатия и без указания варианта. Бюджет усечения в одном месте указан в символах, а в другом — в токенах.

5. **Статистическое утверждение ничем не показано.** Значимость заявлена одним общим парным t-тестом при $p < 0.01$ для горизонтов ≥ 50 — без значений по отдельным сравнениям и без поправки на множественные сравнения.

6. **Незамеченный нуль.** В одной конфигурации с открытыми весами на самом длинном из приводимых горизонтов рантайм со структурированным состоянием и стабильная базовая линия набирают идентично. Таблица это число содержит; текст его не обсуждает.

7. **Авторитет над числом шагов.** Горизонт — это подсчёт внешне генерируемых событий, и ни одно условие не варьирует, кто управляет числом шагов. Это свойство разобрано в §6, и оно — самое значимое из всех различий между той оценкой и нашей.